Booleans and Asking Questions ❓

Christopher Martin - *Bowdoin College*
c.martin@bowdoin.edu

Booleans

A boolean or `bool` is a primitive data type that is only represented by either `True` or `False`.

In Python, they are always capitalized as `True` and `False`.

```python
# Assigning Boolean literals to variables
is_snowing: bool = True
is_sunny: bool = False

print("Is it snowing? ⛄", is_snowing)
print(f"Is it sunny? 🌞 {is_sunny}")
```

```
Is it snowing? ⛄ True
Is it sunny? 🌞 False
```

Okay, but how often will I use `True` or `False`?

By themselves? Probably not very often.. However, `True` and `False` are used heavily because they are the result of a new type of expression!

Relational Operators

Relational operators allow us to ask basic questions about the state of our program's data. Here are the questions we can ask:

Human Operator	Python Symbol	Example Expression
Equal to	`==`	`5 == 5.0`
Not equal to	`!=`	`True != False`
Greater than	`>`	`7.0 > 3`
Less than	`<`	`"A" < "B"`
Greater than or equal to	`>=`	`4 >= 4`

= vs ==

There are two ways we use the *"equals"* operator.

```
x = 5
```

`=` tells the computer to set the value in `x` equal to `5`.
We call this the assignment operator!

```
x == 5
```

`==` asks the computer to check if `x` is equal to `5`.
We call this the equal-to operator!

It is critical that you do not mix these up!

Relational Operators

The result of these relational expressions will be a `bool` value.

This allows us to store the result of asking a question and potentially use that answer later in our program.

```python
word_1: str = "Apple"
word_2: str = "Banana"

in_order: bool = word_1 < word_2

print(f"{word_1} comes before {word_2}? {in_order}")
```

```
Apple comes before Banana? True
```

Okay, so we can get meaningful `bool` values...

But what can we actually do with them?

The `if` Statement

An `if` statement, also called a conditional statement, allows us to ask questions about the state of data and *"conditionally"* execute code based the result.

`if` statements can evaluate any `True` or `False` expression!

The indented code under an `if` statement only executes if the expression (called the condition) evaluates to `True`. If the condition is `False`, the code doesn't run!

```
1  temperature: float = 31.5 # Try changing the temp to see what happens!
2
3  if temperature < 32.0:
4      print("It is freezing! 🥶")
5      print("Maybe it will snow! ⛄")
6  print("Not inside of the if! 🤪")
```

```
It is freezing! 🥶
Maybe it will snow! ⛄
```

The `else` Clause

We can optionally add an `else` clause to `if` statements that defines alternative code to runs in the event that the `if` statement's condition evaluates to `False`.

Each `if` statement can only have one `else` clause.

An `else` must be at the same indentation level as its corresponding `if` statement.

```
1  password: str = "1234"
2
3  if password == "1101S26":
4      print("Welcome to our CodeRunner! ✅")
5  else:
6      print("That password is incorrect! ❌")
7      print("Please try again!")
```

```
That password is incorrect! ❌
```

Nesting

We can nest conditional statements inside of other conditional statements!

We can also put conditional statements inside of functions!

```python
def check_number(num: int) -> str:
    if num == 0:
        return f"{num} is zero!"
    else:
        if num > 0:
            return f"{num} is positive!"
        else:
            return f"{num} is negative!"

print(check_number(10))
print(check_number(-5))
print(check_number(0))
```

```
10 is positive!
-5 is negative!
0 is zero!
```